

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

SCHAAB VALENTINA

TP1 - INTRODUCCIÓN A LA WEB - ARQUITECTURA Y PROTOCOLOS

INTRODUCCIÓN: ELEMENTOS E HISTORIA DE LA WWW

1. ¿CÓMO SURGIÓ EL SISTEMA DE DOCUMENTOS DISTRIBUIDOS WWW?

El WWW surgió en el CERN, a partir de una propuesta de Tim Berners-Lee llamada "Information Management: A Proposal". **La idea era crear un sistema de hipertexto distribuido que permitiera relacionar y consultar información almacenada en diferentes computadoras a través de una red.**

El objetivo era solucionar el problema de compartir información entre científicos de distintas universidades e instituciones. La propuesta combinó hipertexto, computadoras y redes de datos para formar un sistema global de información.

2. ¿EN QUÉ AÑO SE CREO LA WWW? ¿QUIÉN LA CREÓ? ¿CUÁL FUE EL PROPÓSITO INICIAL?

La WWW fue creada en 1989 por Tim Berners-Lee, mientras trabajaba en el CERN.

Su propósito inicial era permitir el intercambio automático y sencillo de información entre científicos y universidades de todo el mundo, mediante documentos de hipertexto conectados entre sí.

(1989 la propuesta/invencción. En 1990 se implementaron el primer servidor y navegador, y en 1991 el software comenzó a difundirse por Internet.)

3. ¿SOBRE QUÉ RED FUNCIONABA? ¿CUÁLES FUERON LOS COMPONENTES INICIALES? WWW EN LA ACTUALIDAD

La WWW funcionaba sobre Internet. Internet es la infraestructura/red, mientras que la WWW es un servicio que utiliza esa red para acceder e intercambiar documentos.

Componentes iniciales:

HTML → lenguaje para estructurar los documentos.

HTTP → protocolo para la comunicación entre cliente y servidor.

URL/URI → mecanismo para identificar y localizar los recursos.

Servidor Web → almacena y ofrece los documentos.

Navegador Web (cliente) → permite acceder y visualizar los documentos.

WWW en la actualidad:

La Web evolucionó desde un sistema pensado principalmente para compartir documentos científicos hasta una plataforma global para sitios web, aplicaciones web, comercio electrónico, redes sociales, multimedia y servicios online. Se basa en estándares abiertos, coordinados principalmente por el W3C, y funciona sobre Internet.

4. NOMBRE Y DESCRIBA LAS DISTINTAS ETAPAS DE EVOLUCIÓN EN LA WEB.
REALICE UNA CRONOLOGÍA DE TECNOLOGÍAS QUE SURGIERON EN LA WEB
HASTA LA ACTUALIDAD.

La evolución de la Web suele dividirse en Web 1.0, Web 2.0 y Web 3.0, aunque actualmente el W3C aclara que técnicamente la Web no tiene versiones formales; estos nombres sirven para describir etapas o tendencias de evolución.

(W3C (World Wide Web Consortium) es una organización internacional que desarrolla y mantiene estándares para la Web.)

Web 1.0 – Web estática

Década de 1990

Fue la primera etapa. La Web se utilizaba principalmente para publicar y consultar información. Las páginas eran mayormente estáticas y el usuario tenía un papel pasivo: podía leer la información, pero no modificarla ni interactuar demasiado con ella.

Tecnologías principales: HTML, HTTP, URL/URI, Servidores y navegadores Web

El W3C identifica justamente HTML, URI y HTTP como la arquitectura inicial de la Web.

Web 2.0 – Web interactiva y social

Década de 2000

La Web pasa de ser principalmente de lectura a ser de lectura y escritura. Los usuarios pueden generar contenido, interactuar, comentar, compartir información y utilizar aplicaciones directamente desde el navegador.

Aparecen redes sociales, blogs, wikis, aplicaciones Web, contenido generado por usuarios

AJAX, JavaScript avanzado, DOM, CSS, XML/JSON.

AJAX permitió actualizar partes de una página de manera asíncrona sin tener que recargarla completamente.

Web 3.0 – Web semántica

Se busca que la información de la Web sea comprensible y procesable por las máquinas, no solamente por las personas. La idea es que los datos tengan significado y puedan relacionarse entre diferentes sistemas.

Tecnologías asociadas: RDF, OWL, XML, SPARQL, Datos enlazados (Linked Data).

El objetivo es avanzar hacia una Web donde las máquinas puedan interpretar y relacionar los datos disponibles.

CRONOLOGIA DE TECNOLOGÍAS

AÑO	TECNOLOGIA/ACONTECIMIENTO	IMPORTANCIA
1989	WWW	Tim Berners-Lee propone la Web
1990	HTTP, HTML, URL	Primer prototipo de la Web
1991	1° Servidor/navegador web	La Web comienza a difundirse
1994	W3C	Se crea el organismo encargado de promover estándares Web
1996	CSS	Permite separar presentación y estructura
1997-98	HTML 3.2 / HTML 4	Evolución del lenguaje HTML
1998	XML	Estructura e intercambio de datos
1998-2000	DOM	Manipulación dinámica de documentos
2000	XHTML	Reformulación de HTML utilizando XM
2000s	WEB 2.0, AJAX	Web dinámica, interactiva y participativa.
2000s	RDF / OWL	Desarrollo de la web semántica
2014	HTML5	Web moderna con multimedia y aplicaciones web.
Actualidad	HTML, CSS, JAVASCRIPT, APIS WEB, WEBASSEMBLY, WEBRTC, etc.	Aplicaciones web completas, multimedia, tiempo real y multidispositivo.

5. ¿QUÉ ES UN SERVIDOR WWW?

Un servidor WWW (servidor Web) es un programa que recibe solicitudes de los clientes Web y les devuelve los recursos solicitados, como páginas HTML, imágenes, archivos, videos o datos.

Utiliza principalmente el protocolo HTTP/HTTPS para comunicarse con los clientes.

6. ¿QUÉ ES UN CLIENTE WEB? EXPLIQUE DETALLADAMENTE.

Un cliente Web es un programa que permite al usuario acceder a recursos de la Web y comunicarse con servidores Web. Ejemplo: un navegador, como Chrome, Firefox o Edge.

Su funcionamiento básico es:

1. El usuario introduce una URL.
2. El cliente interpreta la URL y determina qué recurso se desea obtener.
3. Se comunica con el servidor correspondiente mediante HTTP/HTTPS.
4. El servidor procesa la solicitud y devuelve una respuesta.
5. El cliente recibe los recursos, como HTML, CSS, JavaScript, imágenes, etc.
6. El navegador interpreta esos recursos y construye/renderiza la página que ve el usuario.
7. Si el documento necesita otros recursos, el cliente realiza nuevas solicitudes al servidor.

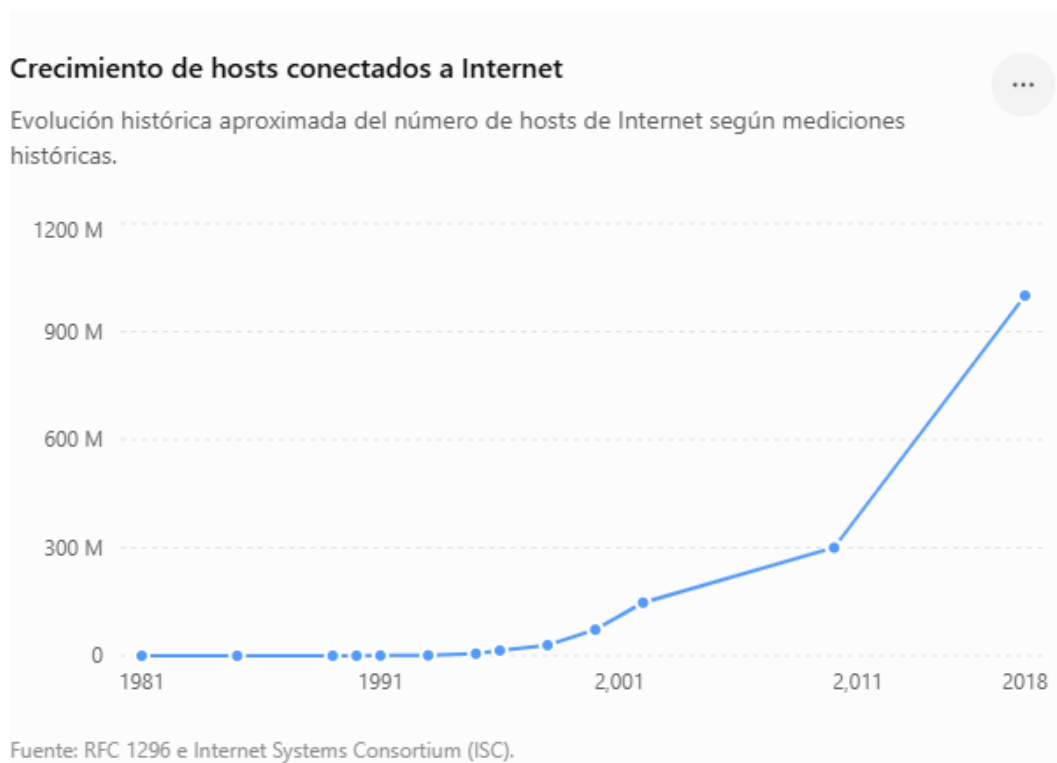
Por ejemplo: al ingresar www.ejemplo.com, el navegador actúa como cliente, solicita la página al servidor y luego interpreta el HTML, CSS y JavaScript para mostrarla.

7. ¿CÓMO ESTÁ COMPUESTO EL SISTEMA WWW ACTUAL?

El WWW actual está compuesto por varios elementos que trabajan conjuntamente:

- **Clientes Web:** principalmente navegadores, que realizan solicitudes y presentan la información al usuario.
- **Servidores Web:** reciben solicitudes y proporcionan recursos o servicios.
- **HTTP/HTTPS:** protocolos utilizados para la comunicación entre clientes y servidores.
- **HTML:** estructura y contenido de los documentos Web.
- **CSS:** define la presentación y apariencia.
- **JavaScript:** permite comportamiento dinámico e interacción.
- **URI/URL:** identifican y permiten localizar recursos.
- **DNS:** permite traducir nombres de dominio a direcciones IP.
- **APIs y servicios Web:** permiten que aplicaciones y sistemas intercambien información.
- **Bases de datos y servidores de aplicaciones:** permiten generar contenido dinámico y procesar la lógica de las aplicaciones.

8. REALICE UN DIAGRAMA DE CRECIMIENTO DE HOST CONECTADOS A INTERNET DESDE SU INICIO HASTA LA ACTUALIDAD.

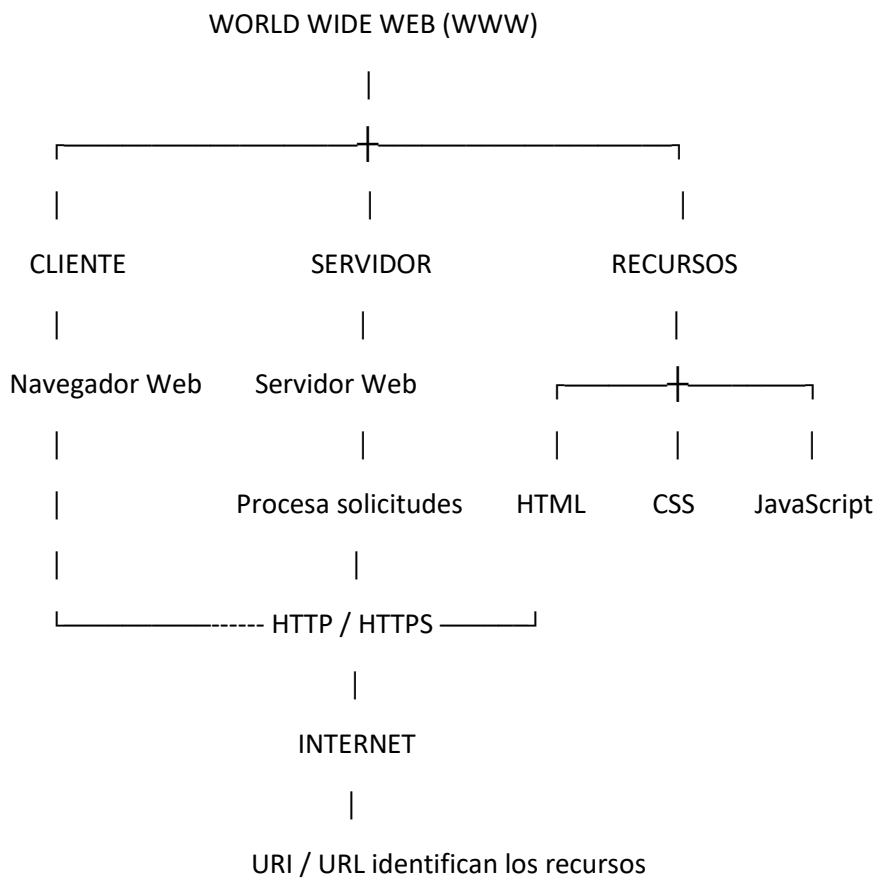


El gráfico demuestra:

- **1981:** apenas había **213 hosts**.
- **1990:** ya había aproximadamente **313.000**.
- **1995:** se superaban los **5,8 millones**.

- **2000:** más de **72 millones**.
- **2002:** más de **147 millones**.
- Desde entonces el crecimiento continuó hasta alcanzar una escala de **cientos de millones y posteriormente más de mil millones de hosts**, aunque las mediciones modernas no son directamente comparables con las históricas. El propio ISC(Internet Systems Consortium) discontinuó su Domain Survey(estudio realizado) porque medía principalmente IPv4 y dejó de considerarlo representativo de Internet completo.

9. REALIZA UN DIAGRAMA CONCEPTUAL DE COMPOSICIÓN DE LA WEB. ¿CUÁLES SON LOS COMPONENTES DE LA WWW?



1. Cliente Web: Es principalmente el navegador. Solicita recursos al servidor. Interpreta HTML, CSS, JavaScript, imágenes, etc. Presenta el resultado al usuario.

2. Servidor Web: Recibe las solicitudes del cliente. Busca, genera y devuelve los recursos solicitados. Se comunica mediante HTTP/HTTPS.

3. HTTP/HTTPS: Son los protocolos de comunicación entre cliente y servidor. HTTP permite solicitar y transferir recursos. HTTPS agrega seguridad mediante cifrado.

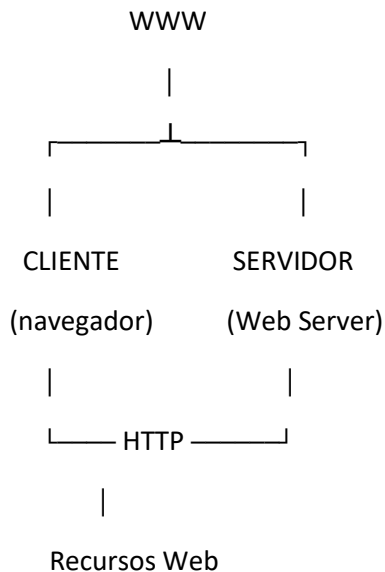
4. URI/URL: Permiten identificar y localizar recursos de la Web.

5. Recursos Web: Pueden ser: HTML, CSS, JavaScript, Imágenes, Videos, Audio, Documentos, Datos, por ejemplo JSON o XML.

6. Internet: Es la red de comunicación sobre la cual funciona la Web.

10. ¿CUÁL ES LA ARQUITECTURA DEL SISTEMA DISTRIBUIDO DE DOCUMENTOS?

La WWW utiliza una **arquitectura cliente-servidor distribuida**.



(HTML, imágenes, etc.)

- **Cliente:** solicita los documentos o recursos.
- **Servidor:** almacena o genera los documentos y responde a las solicitudes.
- **HTTP/HTTPS:** permite la comunicación entre ambos.
- **URI/URL:** identifica cada recurso.
- **Internet:** proporciona la red sobre la cual se comunican.

Es distribuido porque los documentos y recursos no están necesariamente en una única computadora, sino que pueden estar distribuidos entre muchos servidores ubicados en diferentes lugares de la red.

11. ¿CÓMO FUNCIONA EN SISTEMA WWW?

El funcionamiento básico es:

- 1) El usuario introduce una URL en el navegador.
- 2) El navegador determina el servidor al que debe conectarse, utilizando DNS para obtener su dirección IP.
- 3) El cliente establece una conexión con el servidor.
- 4) El navegador envía una solicitud HTTP/HTTPS.
- 5) El servidor procesa la solicitud y busca o genera el recurso solicitado.

- 6) El servidor devuelve una respuesta HTTP/HTTPS, que contiene el recurso y otra información.
- 7) El navegador interpreta el contenido recibido, por ejemplo HTML, CSS y JavaScript.
- 8) Si la página necesita otros recursos (imágenes, CSS, scripts, etc.), el navegador realiza nuevas solicitudes.
- 9) Finalmente, el navegador renderiza la página y la muestra al usuario.

TECNOLOGÍA, SERVIDORES Y CLIENTES

12. ¿QUÉ ES HTML? ¿DE DONDE PROVIENE?

HTML (HyperText Markup Language) es el lenguaje de marcado utilizado para estructurar el contenido de las páginas Web. Permite definir elementos como títulos, párrafos, enlaces, imágenes, tablas, formularios, etc.

Fue creado por Tim Berners-Lee alrededor de 1990, en el CERN, como parte del desarrollo de la World Wide Web. Se basó en ideas del SGML (Standard Generalized Markup Language), adaptadas para el hipertexto.

13. ¿A QUE SE LE LLAMA PÁGINA WEB ESTÁTICA? ¿Y PÁGINA DINÁMICA? ¿Y PÁGINA ACTIVA?

Página Web estática: Es una página cuyo contenido está almacenado previamente en el servidor y se entrega al cliente prácticamente sin modificaciones.

Ejemplo: un archivo index.html que contiene siempre la misma información.

Características: El servidor entrega el archivo, el contenido no cambia según cada solicitud, puede contener HTML y CSS y también JavaScript ejecutado en el cliente.

Página Web dinámica: Es aquella cuyo contenido se genera o modifica en función de una solicitud, datos almacenados, parámetros del usuario, etc.

Por ejemplo, una página que consulta una base de datos y muestra los productos disponibles.

Página Web activa: Una página activa incorpora programas que se ejecutan para proporcionar interacción o comportamiento dinámico, generalmente en el lado del cliente.

Un ejemplo es JavaScript, que puede modificar el contenido de la página, validar formularios, responder a eventos, realizar solicitudes al servidor, etc.

14. ¿QUÉ ES UNA URL? ¿QUÉ FUNCIÓN CUMPLE EN LA WEB? ¿QUÉ ES UN SERVIDOR DE ARCHIVOS BASADO EN URL?

Una URL (Uniform Resource Locator) es una dirección que permite localizar un recurso en la Web.

Una URL puede indicar protocolo (https), servidor/dominio (www.x.com) y ruta del recurso (/doc/index.html).

Función: Permite que un recurso pueda ser **identificado y localizado**, indicando al cliente dónde y mediante qué protocolo puede acceder a él.

Servidor de archivos basado en URL: Es un servidor que permite acceder a archivos almacenados en él mediante URLs. El cliente realiza una solicitud utilizando la URL y el servidor devuelve el archivo correspondiente.

15. ¿DÓNDE Y CÓMO SE EJECUTA EL LENGUAJE HTML?

HTML no se ejecuta como un lenguaje de programación.

El archivo HTML normalmente se encuentra en un servidor Web, que lo entrega al cliente mediante HTTP/HTTPS. Luego, el navegador Web del cliente interpreta el HTML y construye la representación visual de la página.

Por lo tanto, **HTML se procesa/interpreta en el cliente, principalmente por el navegador Web**. El servidor normalmente se encarga de almacenar y entregar el documento HTML.

16. ¿QUÉ ES UN LENGUAJE SCRIPT EN LA TECNOLOGÍA WWW?

Un lenguaje Script es un lenguaje de programación utilizado para agregar comportamiento e interacción a las páginas Web.

A diferencia de HTML, que define la estructura del documento, un script permite realizar acciones como validar formularios, modificar elementos de una página o responder a eventos del usuario. Los scripts pueden ejecutarse en el cliente (navegador) o en el servidor, dependiendo de la tecnología. Ejemplo: JavaScript en el navegador.

17. ¿QUÉ ES UN PLUGIN? ¿QUÉ ES UNA COOKIE?

Un **plugin** (complemento) es un componente de software que se incorpora a una aplicación, especialmente al navegador, para agregar funcionalidades que originalmente no posee.

Históricamente se utilizaron plugins como Flash Player, Java Applets o Silverlight para reproducir contenido multimedia o ejecutar aplicaciones dentro del navegador. Hoy tienen mucha menos importancia porque HTML5 y las APIs Web permiten realizar muchas de esas funciones directamente.

Una **cookie** es una pequeña cantidad de información que un sitio Web guarda en el navegador del usuario. El navegador posteriormente puede enviar esa información al servidor en nuevas solicitudes.

Se utilizan, por ejemplo, para mantener una sesión iniciada, recordar preferencias, guardar información de navegación, personalizar contenido.

18. ¿QUÉ ES JAVASCRIPT? ¿DÓNDE Y QUIÉN EJECUTA JAVASCRIPT? ¿CÓMO NACIÓ JAVASCRIPT? ¿QUÉ ES DHTML? ¿Y XHTML? 25. ¿QUÉ ES XML? EXPLIQUE DETALLADAMENTE.

JavaScript es un lenguaje de programación utilizado principalmente para agregar **comportamiento dinámico e interacción a las páginas Web**.

Permite entre otras cosas, validar formularios, responder a eventos del usuario, modificar el contenido de una página, manipular el DOM, realizar solicitudes a servidores y crear aplicaciones Web interactivas.

Tradicionalmente JavaScript se ejecuta **en el cliente**, dentro del **navegador Web**, mediante el motor JavaScript del navegador. Actualmente JavaScript también puede ejecutarse **del lado del servidor**, por ejemplo mediante **Node.js**.

JavaScript fue desarrollado por Brendan Eich en Netscape en 1995. Originalmente se llamó Mocha, luego LiveScript y finalmente JavaScript. Fue creado para agregar interactividad y comportamiento a las páginas Web desde el lado del cliente.

DHTML (Dynamic HTML) no es un lenguaje independiente. Es un conjunto de tecnologías utilizadas para crear **páginas HTML dinámicas e interactivas**.

Combina principalmente: HTML + CSS + JavaScript + DOM

Esto permite modificar el contenido, estructura y apariencia de una página sin necesidad de volver a cargarla completamente.

XHTML (Extensible HyperText Markup Language) es una reformulación de HTML basada en XML.

Busca que los documentos HTML sigan las reglas más estrictas de XML, por ejemplo: los elementos deben estar correctamente cerrados, los elementos deben estar correctamente anidados, los nombres de etiquetas se escriben respetando mayúsculas/minúsculas, el documento debe estar bien formado.

XML (eXtensible Markup Language) es un lenguaje de marcado extensible diseñado para estructurar, almacenar y transportar información.

A diferencia de HTML, cuyo objetivo principal es representar información en una página Web, XML permite que el usuario defina sus propias etiquetas para describir los datos.

Características: Permite crear etiquetas propias, organiza la información de forma jerárquica, es independiente de plataformas y lenguajes de programación, facilita el intercambio de datos entre sistemas diferentes, los documentos deben estar bien formados, puede utilizar esquemas como DTD o XML Schema para definir su estructura.

19. ¿QUÉ SON LOS SERVICIOS WEB? DE UN EJEMPLO DE APLICACIÓN UTILIZANDO SERVICIOS WEB.

Los Servicios Web (Web Services) son mecanismos que permiten que diferentes aplicaciones se comuniquen e intercambien datos a través de una red, independientemente del lenguaje de programación o plataforma utilizada.

Utilizan estándares y protocolos Web para permitir esa comunicación.

Entre las tecnologías y estándares relacionados se encuentran HTTP/HTTPS, XML, SOAP, WSDL, REST y APIs HTTP.

Ejemplo: sistema bancario

Una aplicación de un comercio necesita consultar si una tarjeta tiene fondos suficientes.

Aplicación del comercio

|

| Solicitud HTTP

↓

Servicio Web bancario

|

↓

Base de datos

|

↓

Respuesta del servicio

|

↓

Aplicación del comercio

El comercio no necesita conocer cómo está programado internamente el banco ni acceder directamente a su base de datos. Utiliza el servicio Web mediante una interfaz definida.

Por ejemplo, puede enviar ConsultarSaldo(tarjeta, monto) y recibir { "autorizado": true }

URL'S, DOMINIOS Y DIRECCIONES

20. ¿QUÉ ES UNA URL? ¿QUÉ FUNCIÓN CUMPLE EN LA WEB? ¿CUÁL ES LA ESTRUCTURA DE UNA URL?

Una URL (Uniform Resource Locator) es una dirección que permite localizar un recurso en Internet/Web. Indica cómo acceder al recurso y dónde se encuentra.

Su estructura general es:

esquema://usuario:contraseña@host:puerto/ruta?consulta#fragmento

- Esquema/protocolo: https
- Host: www.ejemplo.com
- Puerto: 8080 (opcional)
- Ruta: /carpeta/pagina.html
- Consulta: ?usuario=10 (opcional)
- Fragmento: #inicio (opcional)

Ejemplo:

https://www.ejemplo.com:8080/carpeta/pagina.html?usuario=10#inicio

21. ¿QUÉ ES UN SERVIDOR DE ARCHIVOS BASADO EN URL?

Es un servidor Web que permite acceder a archivos mediante URLs.

El cliente solicita una URL y el servidor busca el archivo correspondiente y lo devuelve.

Ejemplo: https://www.ejemplo.com/documentos/manual.pdf

Cliente

|

| GET /documentos/manual.pdf

↓

Servidor Web

|

| busca manual.pdf

↓

Archivo enviado al cliente

22. ¿CUÁL ES EL FORMATO DE UNA URL EN IPV6?

Cuando una URL utiliza directamente una **dirección IPv6**, la dirección debe colocarse **entre corchetes []** para distinguir los : de IPv6 de los : utilizados para indicar un puerto.

Ej: http://[2001:db8::1]/index.html

23. ¿QUÉ ES UN DOMINIO WWW? ¿QUÉ RELACIÓN TIENE CON UNA URL?

Un **dominio** es un **nombre jerárquico que identifica un sitio o servicio en Internet**.

Por ejemplo: ejemplo.com. En lugar de tener que recordar una dirección IP, podemos utilizar un nombre de dominio.

Relación entre dominio y URL: El dominio puede formar parte de una URL, pero no son lo mismo. La URL completa contiene además el protocolo y la ruta del recurso.

Ejemplo:

https://www.ejemplo.com/productos/celular.html

└──dominio──┘

Dominio = nombre del servidor/sitio.

URL = dirección completa utilizada para localizar un recurso.

24. ¿CUÁL ES LA FUNCIÓN DEL DNS EN LA WEB?

DNS (Domain Name System) es el sistema encargado de **traducir nombres de dominio a direcciones IP**, permitiendo que los clientes encuentren el servidor correspondiente.

Cuando escribís una URL en el navegador, este necesita conocer la dirección IP del servidor para poder comunicarse con él. DNS realiza esa resolución. También puede realizar la resolución inversa, es decir, obtener un nombre asociado a una dirección IP.

Ejemplo:

www.ejemplo.com

↓

DNS

↓

203.0.113.10

PROTOCOLOS UTILIZADOS EN LA WWW

25. ¿CUÁL ES LA RELACIÓN ENTRE HTML, HTTP Y HTTPS?

Son tecnologías diferentes que trabajan juntas en la Web:

HTML → define la estructura y contenido de un documento Web.

HTTP → protocolo que permite transferir recursos entre cliente y servidor.

HTTPS → es HTTP protegido mediante TLS, proporciona cifrado, autenticación e integridad.

26. ¿CUÁLES SON LAS VERSIONES DE HTTPS Y CUÁLES SUS DIFERENCIAS?

En realidad, HTTPS no tiene versiones propias. HTTPS es HTTP utilizando TLS. Por eso, lo correcto es hablar de las versiones de HTTP y de las versiones de TLS utilizadas para protegerlo.

Las principales versiones modernas de HTTP son:

Versión	Características
HTTP/1.0	Comunicación básica mediante solicitudes y respuestas.
HTTP/1.1	Conexiones persistentes, mejor manejo de caché y varias mejoras respecto de 1.0.
HTTP/2	Multiplexación de solicitudes, compresión de cabeceras y una comunicación más eficiente.
HTTP/3	Utiliza QUIC sobre UDP en lugar de TCP y mejora el rendimiento y manejo de pérdidas de paquetes.

Para HTTPS, las versiones relevantes de TLS incluyen:

- **TLS 1.2** → ampliamente utilizado y considerado seguro.
- **TLS 1.3** → más moderno, reduce la cantidad de pasos necesarios para establecer la conexión y elimina mecanismos criptográficos antiguos.

Importante: SSL 2.0, SSL 3.0 y TLS 1.0/1.1 son protocolos obsoletos y no deberían utilizarse.

27. ¿QUÉ ES MIME?

MIME (Multipurpose Internet Mail Extensions) es un estándar utilizado para indicar el tipo de contenido de un recurso.

Aunque originalmente fue creado para el correo electrónico, HTTP utiliza los MIME types para que el servidor indique al cliente qué tipo de información está enviando.

Se informa normalmente mediante la cabecera HTTP: Content-Type: text/html. Esto permite que el navegador sepa **cómo interpretar el contenido recibido**.

28. EXPLIQUE CÓMO ES UNA SOLICITUD HTTP.

Una solicitud HTTP es el mensaje que el cliente envía al servidor para solicitar un recurso o realizar una operación.

Su estructura básica es: Línea de solicitud, cabeceras, línea en blanco, cuerpo (opcional).

Sus partes:

1. Método: indica qué operación se quiere realizar. (GET)
2. URI/ruta: indica el recurso solicitado. (/index.html)
3. Versión HTTP: (HTTP/1.1)
4. Cabeceras: proporcionan información adicional sobre la solicitud. (Host: www.ejemplo.com
Accept: text/html)
5. Cuerpo: contiene datos enviados al servidor cuando corresponde, por ejemplo, en una solicitud. (POST).

29. ¿CUÁLES SON LOS MÉTODOS DE HTTP? ENUMERE Y DESCRIBA EL FUNCIONAMIENTO DE CADA UNO.

Los métodos HTTP indican qué operación quiere realizar el cliente sobre un recurso.

GET

Solicita un recurso.

Ej: GET /productos.html. Se utiliza principalmente para obtener información.

POST

Envía datos al servidor para que sean procesados.

Por ejemplo, enviar los datos de un formulario:

POST /usuarios

Los datos suelen enviarse en el cuerpo de la solicitud.

PUT

Crea o reemplaza completamente el recurso indicado.

PUT /usuarios/15

Por ejemplo, reemplazar los datos del usuario 15.

PATCH

Realiza una modificación parcial de un recurso.

PATCH /usuarios/15

Por ejemplo, cambiar solamente el teléfono del usuario.

DELETE

Solicita eliminar un recurso.

DELETE /usuarios/15

HEAD

Es similar a GET, pero el servidor devuelve solo las cabeceras, sin el cuerpo del recurso.

Se utiliza, por ejemplo, para comprobar información sobre un recurso sin descargarlo.

OPTIONS

Indica qué métodos u opciones de comunicación están disponibles para un recurso. Es común verlo relacionado con solicitudes CORS.

CONNECT

Establece un túnel hacia el servidor, utilizado principalmente para conexiones mediante proxy.

TRACE

Realiza una prueba de diagnóstico devolviendo la solicitud recibida. Actualmente suele estar deshabilitado por motivos de seguridad.

BIBLIOGRAFIA PRINCIPAL

- W3C – World Wide Web Consortium
- Tanenbaum, Andrew S. & Van Steen, Maarten. *Distributed Systems: Principles and Paradigms*. 2nd Edition. Prentice Hall, 2006.
- Orfali, Robert; Harkey, Dan; Edwards, Jeri. *Cliente/Servidor y Objetos: Guía de supervivencia*. 3.ª edición. Oxford University Press.
- CERN – A Short History of the Web
- Internet Systems Consortium (ISC)
- RFC / IETF – Internet Engineering Task Force